

TURING MACHINE

hghind

$$TM = (\Sigma, K, S_0, H, \Gamma, \delta)$$

Rispetto ai modelli computazionali visti fin ora sono più potenti, in quanto permettono la scrittura, lettura e spostarsi a destra

$$\delta = (K \times H) \times \Gamma \rightarrow \Gamma \times \{\leftarrow, \rightarrow\}$$

Può risolvere qualsiasi problema computazionale.

Lo stato corrente di una TM è detto CONFIGURAZIONE.
Essa è una tuple costituita da:

$$K \times ((\Gamma - \square) \Gamma^*) \cup \{\epsilon\} \times \Gamma \times (\Gamma^* (\Gamma - \{\square\}) \cup \{\epsilon\})$$


stato corrente celle scansionate a sx celle sotto testina celle a dx

La funzione di transizione dunque indica il passaggio da una configurazione all'altra. Questa f è detta TRANS.

Il passaggio da una configurazione all'altra è detto a quella immediatamente successiva è detto yield-to-one

Una sequenza di configurazioni da start ad uno stato finale è detto PATH. La cardinalità della sequenza è il n.

Le TM vengono utilizzate per verificare se un $w \in L$.

In particolare:

- M accetta w contenuta in L iff se esiste un path
 $(s, \underline{w}) \vdash^n (y, w \sqcup)$

- M rifiuta una stringa w iff $(s, \underline{w}) \not\vdash^n (y, w \sqcup)$

M decide L iff $\forall w \in \Sigma^*$

- M accetta w if $w \in L$

- M rifiuta w if $w \notin L$

Dunque un $L \in D$ iff $\exists TM M$ che lo decide.

Ci sono però classi linguaggi det. SD .

$\rightarrow$ semidecidibile L se e solo se, $\forall$ stringa $\in \Sigma^*$

- Se $w \in L$ M accetta

- Se $w \in L$ M non accetta $\leftarrow$ rifiuta

$\leftarrow$ o rigetto.

Hence $L \in SD$ iff $\exists TM M$ che lo semidecide

Il complemento di SD è $coSD$.

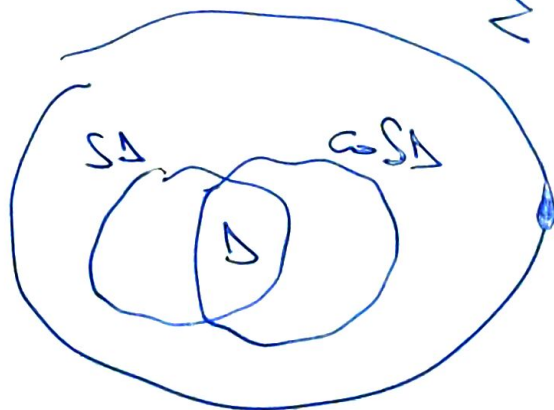
$L \in coSD$ iff $\exists TM$ che semidecide $\bar{L}$

Hence:

$\forall w \in \Sigma^*$

M accetta se $w \notin L$

M non accetta se $w \in L$.



TM come TRANSDUCER.

Se il compito di un TM è quello di eseguire una funzione ovvero, data una stringa in input essa esegue un'operazione in output una nuova stringa w' .

$$(S, w) \vdash^* (h, w')$$

Dunque data $f: \Sigma^* \rightarrow \Sigma^*$ un TM accetta f se:

- Se w è un input su f , allora $M(w) = f(w)$ M si arresta
- Altrimenti: $M(w)$ non si arresta

f è detta ricorsiva ^{computabile} ~~enumerabile~~ se e solo se $\exists$ TM che accetta f .

MULTITAPE

Sono un'estensione delle TM.

E' un caso particolare in cui abbiamo a disposizione più teste.

Una configurazione ~~data~~ è composta da una k -tupla $(s_1, t_1, \dots, t_k)$
STATO INIZIALE

$(s_0, w, D, D, \dots, D)$

STATO FINALE

$(s_f, w', D, \dots, D)$

δ

La funzione di transizione è data da

$$\begin{array}{ccc} (K-W, \Gamma_1 & \rightarrow & (K, \Gamma_2, \{\leftarrow, \rightarrow, \uparrow\}, \\ & \Gamma_2 & , \Gamma_2, \{\leftarrow, \uparrow, \rightarrow\} \\ & \Gamma_n & , \Gamma_n, \{\leftarrow, \rightarrow, \uparrow\}) \end{array}$$

NTM e TM.

Per ogni NTM esiste un'equivalente TM. Dunque non aggiunge potere. Se ~~NTM~~ NTM(w) si arresta allora anche TM(w) si arresta.

La differenza è data dal numero di passi.

Se NTM impiega N allora TM n^2

WWT?

MTN A TM

Si trova un modo per codificare MTN a TM.
ogni tape viene rappresentata da due tracce, una per il
contenuto e una per la testina.

Per far ciò abbiamo bisogno di un alfabeto più grande.

$$\Sigma' = \Sigma \cup (\Gamma \times \Sigma \cup \{0, 1\})^*$$

SIMULAZIONE.

Per simulare MTN a TM si prosegue così:

1- Si prende la stringa in input sulle MTN e viene copiata in TM.
ponendo 1 sotto all'estremo sx della prima tape

- 2.1 Si scannerizza verso sx per trovare lo stato corrente,
e allora a dx.
- 2.2 Si scannerizza verso dx e si aggiornano i dati ^{sulle tape}
in base a δ
- 2.3 torna a 2e

3- Viene controllato tutto insieme la prima traccia.

1 - $|w|$ numero di steps.

$$\left. \begin{array}{l} 2.1 \quad 2 \cdot (|w| + n) \\ 2.2 \quad 2 \cdot (|w| + n) \\ 2.3 \quad |w| + n \\ 3 \quad |w| + n \end{array} \right\} n \text{ volte } \rightarrow O(n^2) \quad \left. \vphantom{\begin{array}{l} 2.1 \\ 2.2 \\ 2.3 \\ 3 \end{array}} \right\} O(n^2)$$

RAN

Una RAN è una struttura di stile di memoria del PC.
Immaginiamo che ~~sia~~ sia infinite funzione nel seguente modo:

Inanzitutto bisogna dire che una cella è divisa in due;

- istruzioni
- dato su cui

La RAN ~~porta~~ con 6 registri



INP → inat

OUT → output program,

PC → indicare indirizzo cella da eseguire,

IR → istruzione da eseguire

AR → indirizzo di memoria del dato su cui eseguire

ACC → accumulatore

Ogni qualvolta viene eseguito un programma avviene questo:

- si parte con 0 nel PC.
- viene inserito l'ist in IR e dato in AR
- finché non si trova ist di stop: esegui ist l'uscita di ASS.

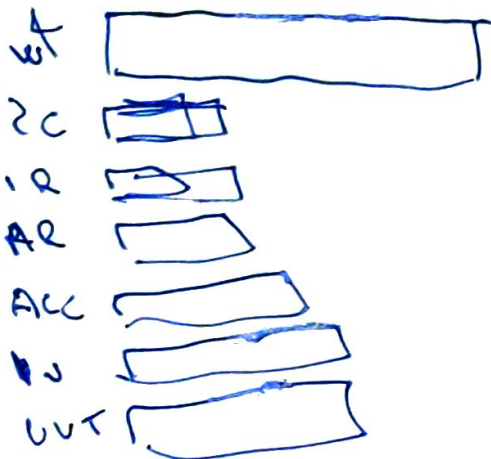
In realtà una RAN è finita dunque le ~~parole molto più~~ parole molto più
simile ad una FAN, però se assumiamo che sia infinita

Non essere paragonata ad una RM

SIMULAZIONE RAM in TM. (stesso potere di una TM)

Una RAM può essere simulata da una TM, precisamente da una NTM.

Avremo fornito una 7-tape TM



Inizialmente il programma e l'input vengono messi nel wt.
1- Per prima cosa l'input $\rightarrow$ IN

2- il PC $\Rightarrow$

3- Spostiamo il cursore della wt a sr until trova il val del pc.

4- i primi 4 bit $\rightarrow$ IR

5- 4 bit + length $\rightarrow$ AR

6- PC + 4

7- Cerco l'operazione corrispondente a quella nell'IR

8- Esego l'istruzione - LOAD

ADD

SUB

etc

9- ciclo

Pende $O(n^6)$???

Se pensiamo

UTM

Lo scopo dell'UTM è di avere una macchina che riceva in input una coppia $\langle M, w \rangle$ e dice se essa decide w .

Per fare ciò abbiamo bisogno di codificare una TM il stringe. Si esegue questo:

$$\begin{matrix} a \\ \swarrow \searrow \\ n \end{matrix} \rightarrow \begin{matrix} \text{lettera} \\ \text{bit} \end{matrix}$$

Per codificare stati: Abbiamo bisogno di n bit $\rightarrow n = \log_2 |K|$
codificare alfabeto: n bit $\rightarrow n = \log_2 |\Sigma|$.
fatto ciò, non resta che δ .

δ è codificata gratuitamente poiché: $\rightarrow$ (stato, simbolo, azione, simbolo)

La codifica della nostra TM non sarà altro che la codifica di δ .

Essendo codificate tutte le TM possiamo dire che $\forall$ TM con C.I.

WORK

Lavora come una TM con 3 nastri

1 - $\langle M \rangle + w \rightarrow \langle M \rangle$.

2 - $\Delta \rightarrow \langle M \rangle$

3 - $\Delta \rightarrow 9000$

Viene analizzata 2 per capire in numero di bit per i, k .

Si cerca 3 in 2. Aggiunge 1 e 3

$$O(\overbrace{|M| \times k}^{\text{step}})$$

Non è realmente universale, ma per ogni TM lo è.

Δ & SD .

Dato una ~~TM~~ esse non e' certo che si ferma

Halt ing Problem.

H ~~de~~ e' semi decidibile.

Leggere Teo.

TRouBLE

Se H fosse in Δ allora ogni L sarebbe in Δ

Ogni linguaggio : SD e' anche SD questo per la def stessa
di Δ ed SD . ~~se una macchina~~



$\&$

REDUCTION

1. una macchina π_1 da L_1 a L_2 o una π_2 da L_2 a L_1
 2. allora π_1 risolve P_1 sfruttando π_2 - oracolo

$$P_1 \leq P_2$$

Se $L_1 \leq L_2$ ed $L_2 \in D$ allora $L_1 \in D$

Se $L_1 \leq L_2$ ed $L_1 \notin D$ allora L_2 non è in D .

Il modo più semplice è usare le mapping reduction, particolari
 + Relazione che trasforma istanze di A in istanze di B .

- Dove sempre esisterà una r tale che $\pi_1(w) = \pi_2(r(w))$

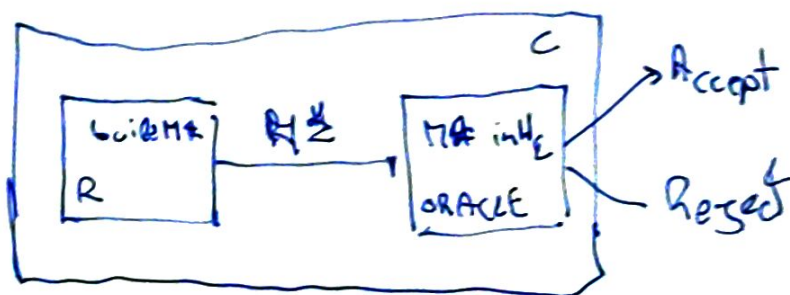
Passi

1- Scegliere linguaggio A_0

2- Descrivere R

3- Descrivere C di R con l'oracolo

4- C decide L_1 se l'oracolo esiste.



leggere $H \in SD$

$H_{ans} \in SD$

$H_{all} \notin D$